Mystery Scroll : Reverse Engineering

Creator: Claudia Varnas Points: 300

Description

This is a simple reverse engineering challenge centered around the xv6 kernel involving system calls. Players will have access to all files that make up the xv6 kernel. It should be noted that the files for this challenge are in their own repo within the CTF's repo.

Github Path: Capture_the_Flame_2026/mystery_scroll

Prompt

Ah– traveller, over here! I've found this secret scroll that was once thought to be lost to time, never to be found again. It contains a set of instructions, but I can't decipher it… it seems to contain words once used in some ancient language. I'm sure if you can figure it out, endless power awaits!

The scroll reads as follows:

"Windows Installation:

- Install WSL2
  In WSL terminal, run:
- sudo apt update -y
- sudo apt install -y build-essential git qemu-system-x86

Mac Installation:

- Install homebrew
  In terminal, run:
- brew install git qemu x86_64-elf-gcc make

Linux Installation:

- sudo apt update -y
- sudo apt install -y build-essential git qemu-system-x86

After installation, in WSL, do: git clone https://github.com/cl-py/xv6-uic-ctf-2026

When inside the xv6-uic-ctf-2026 directory, you may run **make QEMU-NOX** in the terminal to compile. Do **ctrl+a + x** to EXIT.

”

Hints

1. What files are being included in SUSPICIOUS_LETTERS.c?
2. checkword is a system call. Where can we find the function definitions of system calls?
3. sysproc.c

Solution

1. The player is presented with a folder containing files capable of compiling an xv6 kernel. If we enter **ls** in the terminal, we can see the contents.

```
ninth@LAPTOP-5345KR76:~/xv6-uic-ctf-2026$ ls
LICENSE                    _zombie          echo.d           fs.h             kalloc.d     ls.asm       pipe.o       sleeplock.d  sysproc.d    usertests.sym
LucidaSans-Typewriter83    asm.h            echo.o           fs.img           kalloc.o     ls.c         pr.pl        sleeplock.h  sysproc.o    usys.S
Makefile                   bio.c            echo.sym         fs.o             kbd.c        ls.d         printf.c     sleeplock.o  toc.ftr      usys.o
Notes                      bio.d            elf.h            grep.asm         kbd.d        ls.o         printf.d     spinlock.c   toc.hdr      vectors.S
README                     bio.o            entry.S          grep.c           kbd.h        ls.sym       printf.o     spinlock.d   trap.c       vectors.o
SUSPICIOUS_LETTERS.asm     bootasm.S        entry.o          grep.d           kbd.o        main.c       proc.c       spinlock.h   trap.d       vectors.pl
SUSPICIOUS_LETTERS.c       bootasm.o        entryother       grep.o           kernel       main.d       proc.d       spinlock.o   trap.o       vm.c
SUSPICIOUS_LETTERS.d       bootblock        entryother.S     grep.sym         kernel.asm   main.o       proc.h       stat.h       trapasm.S    vm.d
SUSPICIOUS_LETTERS.o       bootblock.asm    entryother.asm   ide.c            kernel.ld    memide.c     proc.o       stressfs.asm trapasm.o    vm.o
SUSPICIOUS_LETTERS.sym     bootblocktmp.o   entryother.d     ide.d            kernel.sym   memlayout.h  real.txt     stressfs.c   traps.h      wc.asm
TAGS                       bootmain.c       entryother.o     ide.o            kill.asm     mkdir.asm    rm.asm       stressfs.d   types.h      wc.c
_SUSPICIOUS_LETTERS        bootmain.o       entryothertmp.o  init.asm         kill.c       mkdir.c      rm.c         stressfs.o   uart.c       wc.d
_cat                       buf.h            exec.c           init.c           kill.d       mkdir.d      rm.d         stressfs.sym uart.d       wc.o
_echo                      cat.asm          exec.d           init.d           kill.o       mkdir.o      rm.o         string.c     uart.o       wc.sym
_forktest                  cat.c            exec.o           init.o           kill.sym     mkdir.sym    rm.sym       string.d     ulib.c       x86.h
_grep                      cat.d            fcntl.h          init.sym         lapic.c      mkfs         runoff       string.o     ulib.d       xv6-64.pdf
_init                      cat.o            file.c           initcode         lapic.d      mkfs.c       runoff.list  swtch.S      ulib.o       xv6.img
_kill                      cat.sym          file.d           initcode.S       lapic.o      mmu.h        runoff.spec  swtch.o      umalloc.c    zombie.asm
_ln                        console.c        file.h           initcode.asm     ln.asm       mp.c         runoff1      syscall.c    umalloc.d    zombie.c
_ls                        console.d        file.o           initcode.d       ln.c         mp.d         sh.asm       syscall.d    umalloc.o    zombie.d
_mkdir                     console.o        forktest.asm     initcode.o       ln.d         mp.h         sh.c         syscall.h    user.h       zombie.o
_rm                        cscope.out       forktest.c       initcodetmp.o    ln.o         mp.o         sh.d         syscall.o    user.ld      zombie.sym
_sh                        date.h           forktest.d       ioapic.c         ln.sym       param.h      sh.o         sysfile.c    usertests.asm
_stressfs                  defs.h           forktest.o       ioapic.d         log.c        pdfmark      sh.sym       sysfile.d    usertests.c
_usertests                 echo.asm         fs.c             ioapic.o         log.d        pipe.c       sign.pl      sysfile.o    usertests.d
_wc                        echo.c           fs.d             kalloc.c         log.o        pipe.d       sleeplock.c  sysproc.c    usertests.o
ninth@LAPTOP-5345KR76:~/xv6-uic-ctf-2026$ vi SUSPICIOUS
```

We can see that there is a suspicious set of files, aptly named SUSPICIOUS_LETTERS.

2. Type vi SUSPICIOUS_LETTERS.c to view the contents of the file.

```
#include "user.h"

int
main(int argc, char *argv[])
{
        int i;

        printf(1, "There's a spot on the scroll where it looks like an inkblot has obscured the words.\n But perhaps this inkblot is merely a disgu
ise...\n");

        printf(1, "%d\n", argc);

        if(argc != 2){
                printf(1, "You notice that the inkblot has darkened. Maybe a step in the wrong direction...\n");
                exit();
        }

        checkword(argv[1]); // This should work. Someone should check that. Not me though!

        exit();

}
```

There is a call to a function named checkword. Checkword is not defined within SUSPICIOUS_LETTERS.c, meaning it must be coming from the #include “user.h”.

3. Type vi user.h to open the file. checkword's header is listed under "system calls", meaning that this function is actually a system call.

```
#pragma once
#include "types.h"
struct stat;
struct rtcdate;

// system calls
int fork(void);
int exit(void) __attribute__((noreturn));
int wait(void);
int pipe(int*);
int write(int, void*, int);
int read(int, void*, int);
int close(int);
int kill(int);
int exec(char*, char**);
int open(char*, int);
int mknod(char*, short, short);
int unlink(char*);
int fstat(int fd, struct stat*);
int link(char*, char*);
int mkdir(char*);
int chdir(char*);
int dup(int);
int getpid(void);
char* sbrk(uint64);
int sleep(int);
int uptime(void);
int checkword(char*);

// ulib.c
int stat(char*, struct stat*);
char* strcpy(char*, char*);
void *memmove(void*, void*, int);
char* strchr(const char*, char c);
int strcmp(const char*, const char*);
void printf(int, char*, ...);
"user.h" 42L, 901B
```

4. Go back to the terminal and enter **ls** once more. There are a few files obviously related to system calls. The correct file to be looking in is sysproc.c. This contains the bodies of all system calls.

```
ninth@LAPTOP-5345KR76:~/xv6-uic-ctf-2026$ ls
LICENSE                 bio.d           entryother      ide.d           kill.d          mkfs            runoff1         syscall.o       usertests.d
LucidaSans-Typewriter83 bio.o           entryother.S    ide.o           kill.o          mkfs.c          sh.asm          sysfile.c       usertests.o
Makefile                bootasm.S       entryother.asm  init.asm        kill.sym        mmu.h           sh.c            sysfile.d       usertests.sym
Notes                   bootasm.o       entryother.d    init.c          lapic.c         mp.c            sh.d            sysfile.o       usys.S
README                  bootblock       entryother.o    init.d          lapic.d         mp.d            sh.o            sysproc.c       usys.o
SUSPICIOUS_LETTERS.asm  bootblock.asm   entryothertmp.o init.o          lapic.o         mp.h            sh.sym          sysproc.d       vectors.S
SUSPICIOUS_LETTERS.c    bootblocktmp.o  exec.c          init.sym        ln.asm          mp.o            sign.pl         sysproc.o       vectors.o
SUSPICIOUS_LETTERS.d    bootmain.c      exec.d          initcode        ln.c            param.h         sleeplock.c     toc.rtr         vectors.pl
SUSPICIOUS_LETTERS.o    bootmain.o      exec.o          initcode.S      ln.d            pdfmark         sleeplock.d     toc.hdr         vm.c
SUSPICIOUS_LETTERS.sym  buf.h           fcntl.h         initcode.asm    ln.o            pipe.c          sleeplock.h     trap.c          vm.d
TRICKS                  cat.asm         file.c          initcode.d      ln.sym          pipe.d          sleeplock.o     trap.d          vm.o
_SUSPICIOUS_LETTERS     cat.c           file.d          initcode.o      log.c           pipe.o          spinlock.c      trap.o          wc.asm
_cat                    cat.d           file.h          initcodetmp.o   log.d           pr.pl           spinlock.d      trapasm.S       wc.c
_echo                   cat.o           file.o          ioapic.c        log.o           printf.c        spinlock.h      trapasm.o       wc.d
_forktest               cat.sym         forktest.asm    ioapic.d        ls.asm          printf.d        spinlock.o      traps.h         wc.o
_grep                   console.c       forktest.c      ioapic.o        ls.c            printf.o        stat.h          types.h         wc.sym
_init                   console.d       forktest.d      kalloc.c        ls.d            proc.c          stressfs.asm    uart.c          x86.h
_kill                   console.o       forktest.o      kalloc.d        ls.o            proc.d          stressfs.c      uart.d          xv6-64.pdf
_ln                     cscope.out      fs.c            kalloc.o        ls.sym          proc.h          stressfs.d      uart.o          xv6.img
_ls                     date.h          fs.d            kbd.c           main.c          proc.o          stressfs.o      ulib.c          zombie.asm
_mkdir                  defs.h          fs.h            kbd.d           main.d          real.txt        stressfs.sym    ulib.d          zombie.c
_rm                     echo.asm        fs.img          kbd.h           main.o          rm.asm          string.c        ulib.o          zombie.d
_sh                     echo.c          fs.o            kbd.o           memide.c        rm.c            string.d        umalloc.c       zombie.o
_stressfs               echo.d          grep.asm        kernel          memlayout.h     rm.d            string.o        umalloc.d       zombie.sym
_usertests              echo.o          grep.c          kernel.asm      mkdir.asm       rm.o            swtch.S         umalloc.o
_wc                     echo.sym        grep.d          kernel.ld       mkdir.c         rm.sym          swtch.o         user.h
_zombie                 elf.h           grep.o          kernel.sym      mkdir.d         runoff          syscall.c       user.ld
asm.h                   entry.S         grep.sym        kill.asm        mkdir.o         runoff.list     syscall.d       usertests.asm
bio.c                   entry.o         ide.c           kill.c          mkdir.sym       runoff.spec     syscall.h       usertests.c
```

5. We can find checkword's code at the very bottom. We see that it takes in user input through argstr, and then does a comparison on the user input with confuse1[]. If the strings are the same, the function prints confuse2, which is the flag.

```
int
sys_checkword(char*)
{
        char *buf; // user input.
        const char confuse1[] = "ilovecs461!";

        if (argstr(0, &buf) < 0){
                return -1;
        }

        if (strncmp(buf, confuse1, strlen(buf)) == 0){
                cprintf("%s\n", confuse2);
        }

        return 0;
}
"sysproc.c" 110L, 1340B
```

6. To solve this, we run **make qemu-nox** and type **SUSPICIOUS_LETTERS ilovecs461!**

```
$ SUSPICIOUS_LETTERS ilovecs461!
There's a spot on the scroll where it looks like an inkblot has obscured the words.
 But perhaps this inkblot is merely a disguise...
woosh! the scroll bursts into flames and reveals the magic spell... flame{leylines}
```

Flag

flame{leylines}